

Technical Blueprint: Admission Module (PoC / MVP)

1. Objective

The goal is to build a highly scalable, automated Admission Module that captures student leads, verifies documents directly from government databases, applies reservation rules (Seat Matrix), and seamlessly converts admitted applicants into enrolled students within a Modular Monolith architecture.

2. Core Features

A. DPDP-Compliant Registration (Top of Funnel)

- **Feature:** User signup with Email/Mobile OTP.
- **Compliance:** Following the strict mandates of India's Digital Personal Data Protection (DPDP) Act 2023, the system will force users to accept a clear "Notice and Informed Consent" form before collecting any Personally Identifiable Information (PII). This digital consent will be logged in the database with a timestamp and the user's IP address.

B. DigiLocker Document Vault (Paperless Verification)

- **Feature:** "Fetch from DigiLocker" functionality during application form filling.
- **Execution:** We will integrate the OAuth 2.0 flow via API Setu. When the applicant authorizes the application, our system will call DigiLocker's 'Pull URI Request API' to fetch their 10th/12th mark sheets in machine-readable XML or Base64 PDF format. This reduces manual verification time and eliminates the risk of forged documents entirely.

C. Dynamic Seat Matrix & Merit Engine

- **Feature:** Tracking reservations and available college seats.
- **Execution:** The system will utilize a database-driven Seat Matrix that balances merit scores with category reservations (General, SC, ST, OBC). If a specific quota (e.g., SC) is completely filled, the system will automatically block further allocations in that category.

D. State-Machine Workflow

- **Feature:** Live tracking and auditing of applications.
- **Execution:** Every application form will follow a definitive lifecycle, acting as a finite-state machine: **Draft** $\rightarrow$ **Submitted** $\rightarrow$ **Doc_Verified** $\rightarrow$ **Seat_Offered** $\rightarrow$ **Fee_Paid**. Every status change will be recorded in a dedicated audit table to maintain a historical trail.

3. Database Architecture

To ensure high performance, we will avoid using random **UUIDv4** keys. Instead, we will use **UUIDv7**. Because **UUIDv7** is time-ordered, it prevents B-Tree index fragmentation in PostgreSQL, keeping insert queries heavily optimized and cache-efficient at a large scale.

Your ORM (Prisma/Drizzle) will consist of these 6 core tables for the Admission Module:

1. **admission_leads**: The applicant's initial data (Name, Email, Phone) and the **dmdp_consent_logged_at** (timestamp).
2. **applications**: The core form data (Application Number, Program ID, Reservation Category, and Current Status).
3. **applicant_academic_records**: Past education details (10th/12th scores, Entrance Exam Ranks) used by the merit engine.
4. **applicant_documents**: AWS S3/Cloud storage URLs and an **is_digilocker_verified** boolean flag.
5. **seat_matrix_quotas**: The total allocated seats and the category-wise filled seats count for each academic program.
6. **application_status_history**: The audit trail table tracking **old_status**, **new_status**, **changed_by_user_id**, and **timestamp**.

ADMISSION MODULE – DATABASE SCHEMA & SYSTEM WORKFLOW

Complete Schema, Relationships, Workflow & Integration with Student Module

DATABASE SCHEMA (ADMISSION MODULE)

1 admission_leads (Registration & Consent)					
PK	id	UUIDv7			
	tenant_id	UUID (College ID)			
	first_name	VARCHAR(100)			
	last_name	VARCHAR(100)			
	email	VARCHAR(255)			
	phone	VARCHAR(20)			
	is_email_verified	BOOLEAN DEFAULT FALSE			
	ddpp_consent_accepted	TIMESTAMP			
	ddpp_consent_ip	VARCHAR(45)			
	created_at	TIMESTAMP			
	updated_at	TIMESTAMP			

DDPP Act 2023 Compliance:
Data is captured only after explicit consent.

2 applications (Core Admission Form)					
PK	id	UUIDv7			
FK	lead_id	UUID (→ admission_leads.id)			
	tenant_id	UUID			
	program_id	UUID (→ programs.id)			
	application_number	VARCHAR(50) UNIQUE			
	entrance_score	DECIMAL(10,2) NULL			
	applied_category	VARCHAR(50)			
	current_status	VARCHAR(30)			
		(Draft, Submitted, Under Review, Verified, Seat Offered, Fee Paid, Rejected, Cancelled)			
	submitted_at	TIMESTAMP NULL			
	created_at	TIMESTAMP			
	updated_at	TIMESTAMP			

3 applicant_documents (DigLocker Vault)					
PK	id	UUIDv7			
FK	application_id	UUID (→ applications.id)			
	doc_type	VARCHAR(100)			
		(10th Marksheet, 12th Marksheet, Auditor, Migration, etc.)			
	storage_url	TEXT			
	is_diglocker_verified	BOOLEAN DEFAULT FALSE			
	diglocker_doc_id	VARCHAR(255) NULL			
	uploaded_at	TIMESTAMP			
	verified_at	TIMESTAMP NULL			

DigLocker Verified = TRUE
→ No manual verification needed.

4 application_status_history (Workflow / State Machine)					
PK	id	UUIDv7			
FK	application_id	UUID (→ applications.id)			
	old_status	VARCHAR(30) NULL			
	new_status	VARCHAR(30)			
	changed_by_admin_id	UUID (→ users.id) NULL			
	remarks	TEXT NULL			
	created_at	TIMESTAMP DEFAULT NOW()			

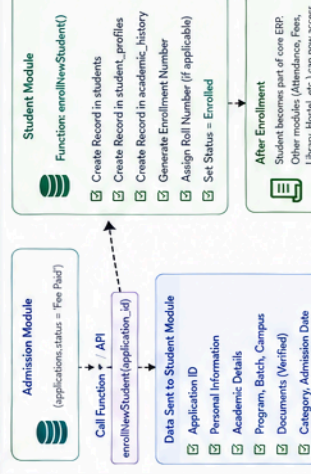
5 seat_matrix (Reservation & Quota Engine)					
PK	id	UUIDv7			
	tenant_id	UUID			
	program_id	UUID (→ programs.id)			
	category	VARCHAR(50)			
		(General, SC, ST, OBC, EWS, etc.)			
	total_seats_allocated	INT			
	seats_filled	INT DEFAULT 0			
	seat_type	VARCHAR(50)			
		(Government Quota, Management Quota, NRI, etc.)			
	academic_year	VARCHAR(20)			
		(e.g. 2024-25)			
	UNIQUE	(program_id, category, academic_year, seat_type)			

6 admission_payments (Fee Tracking)					
PK	id	UUIDv7			
FK	application_id	UUID (→ applications.id)			
	amount	DECIMAL(12,2)			
	currency	VARCHAR(10) DEFAULT 'INR'			
	payment_gateway	VARCHAR(50)			
		(Razorpay, PayU, etc.)			
	payment_gateway_ref	VARCHAR(255)			
	status	VARCHAR(20)			
		(Pending, Success, Failed, Refunded)			
	paid_at	TIMESTAMP NULL			
	created_at	TIMESTAMP			

ADMISSION WORKFLOW (STATE MACHINE)



INTEGRATION – ADMISSION MODULE ↔ STUDENT MODULE



KEY FEATURES / WOW FACTORS

- DDPP Act 2023 Compliant**: Consent required before data capture.
- DigLocker Integration**: Fetch & verify govt. documents digitally.
- Reservation & Quota Engine**: Real-world Indian admission workflow.
- Workflow State Machine**: Full audit trail, no manual overwrites.
- Modular & Scalable**: Clean separation: Admission → Student.
- API First Architecture**: Easy integration with future modules.

4. The Integration "Handshake" (Connecting to the Student Module)

In enterprise Modular Monoliths, one module's domain must never directly write to or query another module's database tables. Therefore, the Admission Module will never insert data directly into the `students` table. There will be a strict API contract between you and your partner.

The Workflow:

1. When the applicant's process is fully complete, the Admin clicks "Confirm Admission".
2. The Admission Module's Backend will send a JSON payload to the Student Module's Backend via an internal REST API call (e.g., `POST /api/v1/students/enroll`).

JSON Payload Contract (Example):

JSON

```
{
  "application_number": "APP-2024-8799",
  "program_id": "BTECH-CS",
  "personal_details": {
    "full_name": "Rahul Sharma",
    "email": "rahul@example.com",
    "phone": "9876543210",
    "category": "General"
  },
  "compliance_data": {
    "abc_id": "1234-5678-9012",
    "digilocker_verified": true
  }
}
```

3. As soon as the Student Module receives this verified data, it will independently generate a new, permanent `Enrollment Number`, insert the data into its own `students` and `student_profiles` tables, and return a `201 Created` response.

5. Developer Workflow (Parallel Engineering)

To work simultaneously without stepping on each other's toes or causing code conflicts, we will follow this workflow:

- **Database Branching:** Using a serverless Postgres provider like Neon, Developer A (Admissions) and Developer B (Student) will create their own isolated database branches (e.g., `feat-admission`, `feat-student`). This allows both of you to run schema migrations without breaking the other person's database.
- **Independent Schemas:** Developer A will exclusively work on the 6 admission tables mentioned above.

- **Mock Testing:** While the Admission UI and logic are being built, Developer B (Student Module) does not have to wait. They can use Postman to send mock JSON payloads to their enrollment API to test the student creation logic.
- **Deployment CI/CD:** GitHub Actions will be configured so that whenever code is merged into the main branch, the pipeline automatically runs the database migration scripts (`npx prisma migrate deploy`) *before* deploying the new application code.